LSTM Networks from the Ground Up

Recurrent networks, vanishing gradients, gates and the cell state, with runnable code

---

A 20-slide tutorial • companion code: `lstm_numpy.py`, `lstm.py`, `data.py`, `train.py`, `experiments.py`

Sequence data: the order carries the information

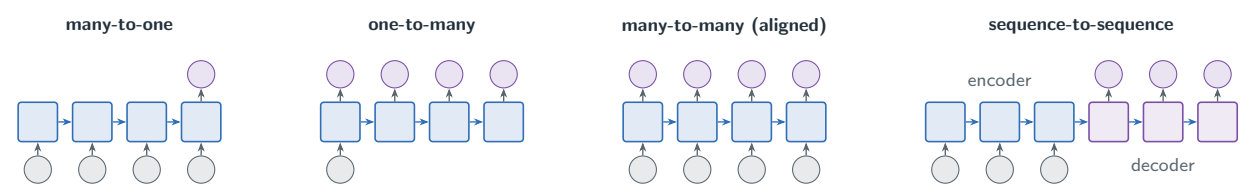
A **sequence** is an ordered list $x_1, x_2, \ldots, x_T$ whose length T may differ from example to example: characters of a text, monthly measurements, audio samples.

Order matters. "dog bites man" and "man bites dog" contain the same words. Shuffle a time series and its histogram is unchanged, but trend and cycles are gone.

An MLP needs a fixed-size input and separate weights for every position. A **recurrent network** reads one element at a time and keeps a **state**: a summary of what it has seen so far, updated by the same rule at every step.

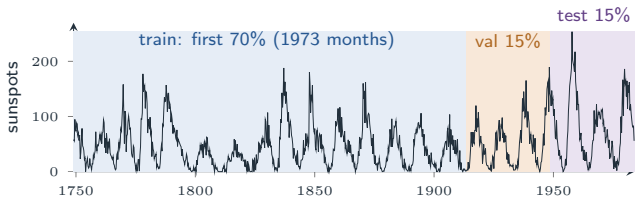
Three measured tasks used throughout

- **Adding problem** (synthetic): find two marked numbers in a sequence of length T and output their sum. Tests long-range memory.
- **Sunspots**: 1973 + 423 + 424 monthly values, 1749–1983. Forecast the next months.
- **Tiny Shakespeare**: 1.1 million characters. Predict the next character, then generate text.

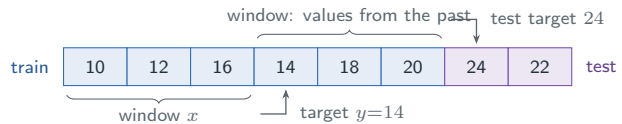


Grey: inputs x_t. Blue: the recurrent cell and its state. Purple: outputs. The same cell is drawn once per time step.

Preparing sequence data: windows, ids, and a split by time

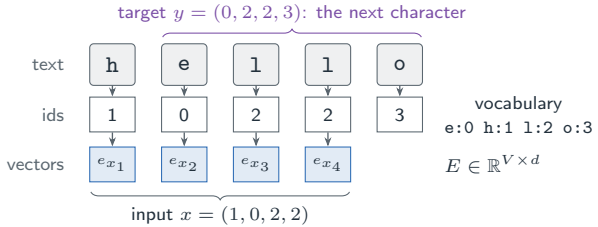


Split by time, never at random. Neighbouring windows share 23 of their 24 values: a random split puts near-copies of every test window into training and lets the model learn from the future.



Scale with training statistics only: $\mu = 15$, $\sigma = 3.42$ from the six training values, so $16 \to (16 - 15)/3.42 = \mathbf{0.293}$. Using all eight values gives $\mu = 17$, $\sigma = 4.58$: the test period has leaked. Sunspots: $\mu = 45.1$, $\sigma = 37.9$ from 1749–1913.

Text: characters (or word pieces) → integer ids → vectors.

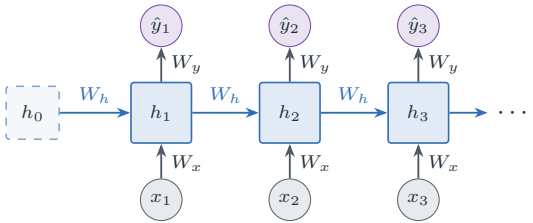


- **One-hot**: id k becomes a vector of V zeros with a 1 at position k.
- **Embedding**: id k selects row k of a learned table E: the same as one-hot times a weight matrix, without the wasted multiplications. Here $V = 65$, $d = 32$.
- The target is the input shifted by one position.

Checklist (`data.py`)

Split first, by position in time; then fit scalers on the training part. A test window may *read* earlier months (the past is known when forecasting); its *target* must lie in the test period.

The simple recurrent network (Elman RNN)



One rule, applied at every step $t = 1, \dots, T$

$$h_t = \tanh\big(W_x\, x_t + W_h\, h_{t-1} + b\big), \qquad \hat{y}_t = W_y\, h_t + b_y$$

$x_t \in \mathbb{R}^{n_{\text{in}}}$, $h_t \in \mathbb{R}^{n_h}$, $W_x \in \mathbb{R}^{n_h \times n_{\text{in}}}$, $W_h \in \mathbb{R}^{n_h \times n_h}$, $h_0 = 0$.

- The **hidden state** h_t is the network's only memory: everything it knows about $x_1..x_t$ must fit in n_h numbers.
- **Unrolling**: drawn over time, the RNN is a deep network with one layer per time step.
- **Parameter sharing**: every step uses the same W_x, W_h, b. The count, $(n_{\text{in}} + n_h + 1)\, n_h$, does not depend on T; a pattern learned at one position works at all positions [1].

Two steps by hand $(n_{\text{in}} = 1,\ n_h = 2,\ h_0 = 0)$

$$W_x = \begin{bmatrix} 0.5 \\ -1 \end{bmatrix}, \quad W_h = \begin{bmatrix} 0.5 & -0.5 \\ 0.25 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$$

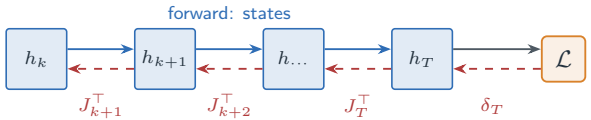
$$\begin{aligned}
x_1 = 1\text{:}\quad & z_1 = W_x \cdot 1 + W_h h_0 + b = (0.5,\ -0.5) \\
& h_1 = \tanh z_1 = (\mathbf{0.462},\ \mathbf{-0.462}) \\
x_2 = 2\text{:}\quad & z_2 = W_x \cdot 2 + W_h h_1 + b \\
& \quad = (1, -2) + (0.462, -0.347) + (0, 0.5) \\
& \quad = (1.462,\ -1.847) \\
& h_2 = \tanh z_2 = (\mathbf{0.898},\ \mathbf{-0.951})
\end{aligned}$$

What to notice

- h_2 depends on x_1 only through h_1: the past reaches the present through the state and nothing else.
- $\tanh$ keeps every state in $(-1, 1)$: the state cannot blow up.
- 8 parameters here: $(1 + 2 + 1) \cdot 2$. A sequence of 2 steps or 2,000 uses the same 8.
- The output layer W_y is an ordinary linear layer, used at every step (many-to-many) or only at $t = T$ (many-to-one).

`walkthrough.py`, section 3, prints these numbers.

Backpropagation through time and the product of Jacobians



Unrolled, the RNN is a deep network and the chain rule applies: **backpropagation through time** (BPTT) [2]. Let $\delta_t = \partial \mathcal{L} / \partial h_t$, with the loss at the last step.

One step back multiplies by one Jacobian

$$J_t = \frac{\partial h_t}{\partial h_{t-1}} = \operatorname{diag}\big(1 - h_t^2\big)\, W_h, \qquad \delta_{t-1} = J_t^\top\, \delta_t$$

$$\delta_k = \big(J_T\, J_{T-1} \cdots J_{k+1}\big)^\top \delta_T$$

$$\frac{\partial \mathcal{L}}{\partial W_h} = \sum_{t=1}^{T} \big(\delta_t \odot (1 - h_t^2)\big)\, h_{t-1}^\top$$

Shared weights: the gradient is a *sum over time steps*; the term linking step k to the loss carries a product of $T - k$ Jacobians.

A product of many similar factors either shrinks or grows. With $\sigma_{\max}$ the largest singular value of W_h and $\tanh' \le 1$:

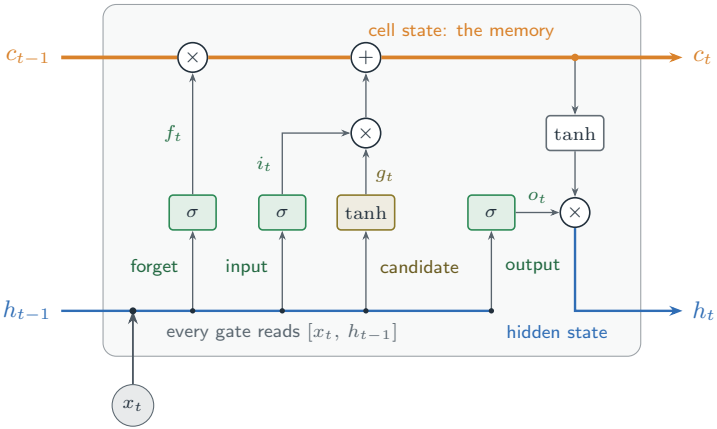
$$\|\delta_k\| \;\le\; \big(\sigma_{\max}(W_h)\big)^{T-k}\, \|\delta_T\|$$

- **Vanishing** ($\sigma_{\max} < 1$, or saturated $\tanh$): the signal from $T - k$ steps back decays exponentially. $0.9^{50} = 0.005$, $0.9^{100} = 3 \times 10^{-5}$. The total gradient is not zero; it is dominated by the recent past, so **long-range dependencies are not learned** [3].
- **Exploding** (needs $\sigma_{\max} > 1$): $1.1^{50} = 117$, $1.1^{100} = 1.4 \times 10^4$. One huge step can wreck the weights. Remedy: clipping (slide 12) [4].

The core difficulty

To store information reliably the state must be stable under small perturbations, which means $\|J_t\| < 1$: exactly the condition for vanishing gradients [3]. Tuning W_h cannot fix this. The architecture must change (slides 6–9).

The LSTM cell: three gates and a separate memory



- A **gate** is a sigmoid layer whose outputs in $(0, 1)$ multiply another vector elementwise: 0 blocks a component, 1 lets it through.
- Two states: the **cell state** c_t stores, the hidden state h_t is what the gates, the next layer and the output see.
- The 1997 LSTM [5] had input and output gates; the forget gate was added in 2000 [6].

The six equations

$$f_t = \sigma\big(W_f\,[x_t, h_{t-1}] + b_f\big) \quad \text{forget}$$

$$i_t = \sigma\big(W_i\,[x_t, h_{t-1}] + b_i\big) \quad \text{input}$$

$$g_t = \tanh\big(W_g\,[x_t, h_{t-1}] + b_g\big) \quad \text{candidate}$$

$$o_t = \sigma\big(W_o\,[x_t, h_{t-1}] + b_o\big) \quad \text{output}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad \text{cell}$$

$$h_t = o_t \odot \tanh(c_t) \quad \text{hidden}$$

$[x_t, h_{t-1}] \in \mathbb{R}^{n_{\text{in}}+n_h}$ is the two vectors stacked; $\sigma(z) = 1/(1+e^{-z})$; $\odot$ is the elementwise product.

Read the cell update as a sentence: keep a fraction f_t of the old memory, and add a fraction i_t of the new candidate g_t.

In one line: stack the four weight matrices into $W \in \mathbb{R}^{4n_h \times (n_{\text{in}}+n_h)}$, compute $z_t = W[x_t, h_{t-1}] + b$ once and split it in four. That is what the code does (slide 15).

One LSTM step by hand

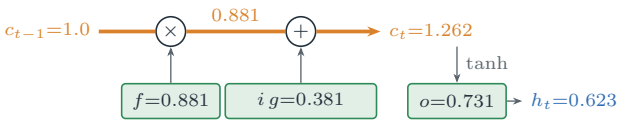
Smallest possible cell: $n_{\text{in}} = 1$, $n_h = 1$. Inputs to this step:

$$x_t = 1, \qquad h_{t-1} = 0.5, \qquad c_{t-1} = 1.0$$

	w_x	w_h	b	$z = w_x x_t + w_h h_{t-1} + b$	value
forget f	0.5	1	1	$0.5 + 0.5 + 1 = 2$	$\sigma(2) = \mathbf{0.881}$
input i	1	−2	0	$1 - 1 + 0 = 0$	$\sigma(0) = \mathbf{0.500}$
candidate g	0.5	1	0	$0.5 + 0.5 + 0 = 1$	$\tanh(1) = \mathbf{0.762}$
output o	1	2	−1	$1 + 1 - 1 = 1$	$\sigma(1) = \mathbf{0.731}$

$$c_t = f\, c_{t-1} + i\, g = 0.881 \cdot 1.0 + 0.500 \cdot 0.762 = 0.881 + 0.381 = \mathbf{1.262}$$

$$h_t = o\, \tanh(c_t) = 0.731 \cdot \tanh(1.262) = 0.731 \cdot 0.852 = \mathbf{0.623}$$



Reading the numbers

- **Forget gate 0.88**: 88% of the old memory survives. The bias $b_f = 1$ contributes half of z_f; without it $f = \sigma(1) = 0.73$.
- **Input gate 0.5** lets half of the candidate 0.762 in.
- The cell state *grew* above 1. It is not squashed, so it can count and accumulate.
- **Output gate 0.73**: h_t is a squashed, scaled view of the memory. A cell can hold a value while $o \approx 0$ hides it.

Next step with $x_{t+1} = 0$: $c_{t+1} = 1.177$, $h_{t+1} = 0.464$. The memory fades slowly at a rate set by f, not by a weight matrix.

The local derivative that matters: along the cell state, $\partial c_t / \partial c_{t-1} = f_t = 0.881$ on the direct path. In an RNN the corresponding factor is $\partial h_t / \partial h_{t-1} = (1 - h_t^2)\, w_h$.

Parameters: 4 rows of $(w_x, w_h, b) = 4\,(1 + 1 + 1) \cdot 1 = 12$.

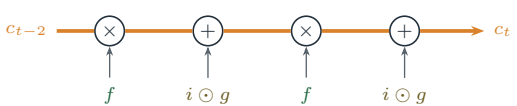
`walkthrough.py`, section 5, prints every number here and checks it against `lstm_numpy.lstm_step` and `lstm.LSTMCellScratch`.

Why it works: the cell state is a conveyor belt

simple RNN: every step multiplies by W_h and squashes



LSTM: the cell state is only scaled by f and added to



Differentiate $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ along the belt:

The direct path has no weight matrix and no squashing

$$\frac{\partial c_t}{\partial c_{t-1}} = \operatorname{diag}(f_t) + (\text{terms via } h_{t-1}), \quad \left.\frac{\partial c_T}{\partial c_k}\right|_{\text{direct}} = \operatorname{diag}\Big(\prod_{t=k+1}^{T} f_t\Big)$$

With $f_t = 1$ and $i_t = 0$ the cell copies itself and the gradient passes back unchanged: the **constant error carousel** of [5]. Residual connections in deep networks rely on the same additive idea.

The decay rate is now learned, per unit and per step:

forget gate f	f^{20}	f^{50}	f^{100}
$\sigma(0) = 0.50$	10^{-6}	9×10^{-16}	8×10^{-31}
$\sigma(1) = 0.73$	0.002	2×10^{-7}	2×10^{-14}
0.99	0.82	0.61	0.37

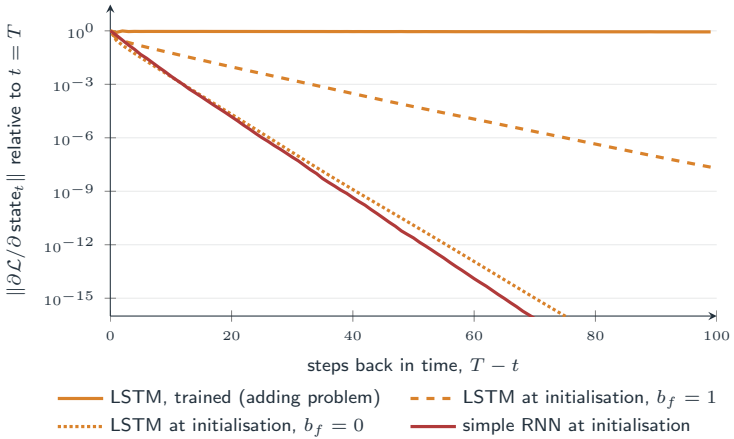
Why the forget-gate bias starts at 1

With all biases 0 a fresh LSTM has $f \approx 0.5$: it forgets half its memory every step and gradients vanish as fast as in an RNN (slide 9). Setting $b_f = 1$ starts the gates mostly open [6,8]. Keras does this by default; PyTorch does not (`set_forget_bias` in `lstm.py`).

Measured, adding problem, $T = 40$, steps until validation MSE < 0.01 for 3 seeds. $b_f = 0$: 500, 800, 700. $b_f = 1$: 500, 600, 500. A modest gain at this length.

Not solved: h_t still feeds the gates through weight matrices, where gradients can explode, so clipping stays (slide 12).

Measured: how far back does the gradient reach?



Measured by `experiments.py grad`: $n_h = 32$, sequences of length 100 from the adding problem, loss at the last step only. State $= h_t$ (RNN) or (h_t, c_t) (LSTM). Initialisation curves: geometric mean of 5 seeds, PyTorch default weights. Trained curve: one model.

gradient norm ratio	20 back	50 back
RNN, init	1.5×10^{-5}	2.4×10^{-12}
LSTM, init, $b_f = 0$	2.1×10^{-5}	1.2×10^{-11}
LSTM, init, $b_f = 1$	9.5×10^{-3}	5.6×10^{-5}
LSTM, trained	9.2×10^{-1}	9.0×10^{-1}

- **RNN**: a straight line on a log axis is exponential decay, as slide 5 predicts.
- **An untrained LSTM also vanishes.** With $b_f = 0$ it is no better than the RNN. The architecture *permits* long memory; it does not guarantee it.
- $b_f = 1$ slows the decay by many orders of magnitude.
- **After training** (1500 steps, test MSE 0.0009) the gradient 100 steps back is still 8.7×10^{-1} of its final value. The most open unit averages $f = 0.999$; the mean over all units is only 0.57.

Gating gives a *learnable* memory span: units that must remember keep $f \approx 1$, the rest stay free to forget.

Variants: counting parameters, GRU, stacked and bidirectional

Counting. Each gate or candidate is one layer on $[x_t, h_{t-1}]$.

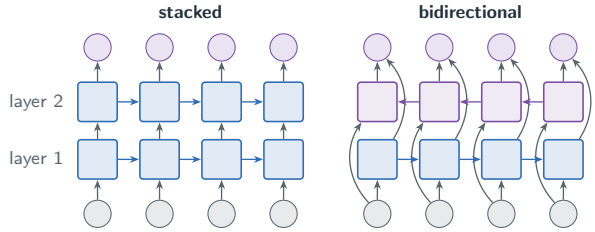
	formula	n_{in}=32, n_h=256	`torch.nn`
RNN	$(n_{\text{in}} + n_h + 1)\, n_h$	73,984	74,240
GRU	$3\,(n_{\text{in}} + n_h + 1)\, n_h$	221,952	222,720
LSTM	$4\,(n_{\text{in}} + n_h + 1)\, n_h$	295,936	296,960

PyTorch keeps two bias vectors per layer (`bias_ih`, `bias_hh`): $4n_h = 1{,}024$ extra for the LSTM. Our `LSTMCellScratch` has exactly 295,936.

GRU: two gates, one state [9]

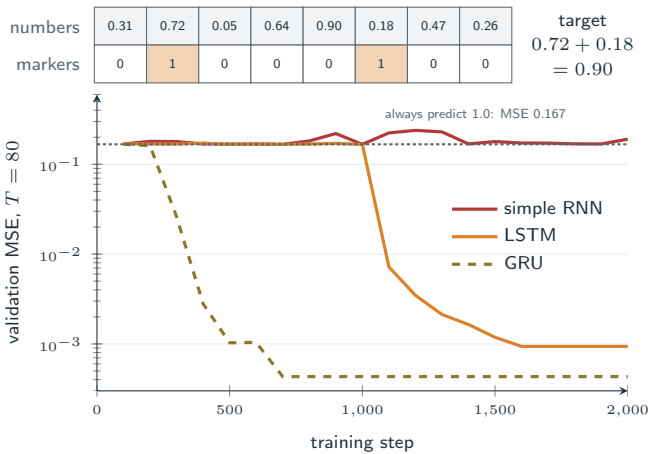
$$z_t = \sigma\big(W_z[x_t, h_{t-1}] + b_z\big) \quad \text{update}$$
$$r_t = \sigma\big(W_r[x_t, h_{t-1}] + b_r\big) \quad \text{reset}$$
$$n_t = \tanh\big(W_n[x_t,\ r_t \odot h_{t-1}] + b_n\big) \quad \text{candidate}$$
$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot n_t \quad \text{state}$$

The update gate is forget and input gate in one ($f = z$, $i = 1 - z$); no separate cell, no output gate. Neither GRU nor LSTM wins consistently [7,10].



- **Stacked**: layer ℓ reads the hidden states of layer $\ell - 1$ as its input sequence. Depth in space on top of depth in time; 2–4 layers were typical [12,15] (`--layers`).
- **Bidirectional** [11]: a second RNN (purple) reads right to left; the two hidden states are concatenated, so each output sees past *and* future. Doubles the parameters (`--bidirectional`).
- Bidirectional models suit tagging and classifying complete sequences. They cannot forecast or generate: at step t the future does not exist yet.
- **When GRU?** Smaller models, less data, tight latency budgets. **When LSTM?** When a separate, unsquashed memory helps: counting, long gaps.

Measured: does gating matter? The adding problem



Seed 0 of each model. A run stops once validation MSE < 0.001.

The task [5]: output the sum of the two marked numbers. One marker lies in each half of the sequence, so the gap is about $T/2$ steps.

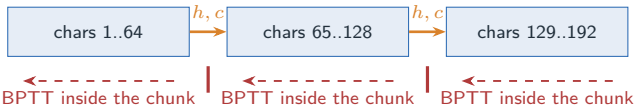
solved in ≤ 2000 steps	T=10	T=20	T=40	T=80
simple RNN	3/3	0/3	0/3	0/3
LSTM	3/3	3/3	3/3	3/3
GRU	3/3	3/3	3/3	3/3
median steps until validation MSE < 0.01				
simple RNN	1000	–	–	–
LSTM	300	400	500	1000
GRU	200	200	300	300

"Solved": test MSE < 0.01 on 2,000 fresh sequences. 3 seeds per cell; $n_h = 32$, Adam, $\eta = 0.01$, batch 32; same code and budget for all.

- The simple RNN manages only $T = 10$. Both gated networks solve every length; the plateau before the drop lengthens with T.
- **Honest note**: here the GRU learns faster than the LSTM. Gating matters, not the particular gated cell.

Training I: truncated BPTT, carried state and gradient clipping

state *values* are carried forward; the gradient is cut (`detach`)



Truncated BPTT. A text of a million characters is one sequence; backpropagating through all of it for each update is unaffordable. Cut it into chunks of T steps: the forward state crosses chunk borders, gradients stop there. Dependencies longer than T can be *used* but are not *learned* directly.

Initial state. $h_0 = c_0 = 0$ at the start of an independent sequence (a window, a document). Along a stream, each chunk starts from the previous final state (`stateful=True` in `fit`).

Gradient clipping by norm [4]

$g \leftarrow g \cdot \min\big(1,\ \tau/\|g\|\big)$. E.g. $g = (3, 4)$, $\tau = 1$: $\|g\| = 5$, so $g \to (0.6,\ 0.8)$. Direction kept, length capped at τ; all parameters are scaled together (`clip_grad_norm_`).

Honest note: the divergence on the right needed plain SGD and a large η. With Adam we saw gradient spikes but no divergence. We keep $\tau = 1$ everywhere as cheap insurance.

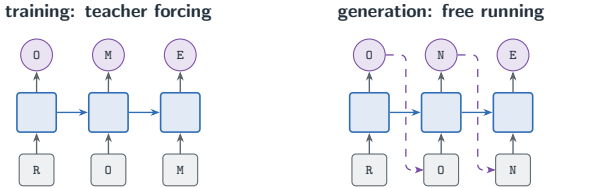


Measured: simple RNN, $n_h = 128$, Shakespeare characters, plain SGD, $\eta = 2$, chunks of 100. Curves and the first two table columns: seed 0.

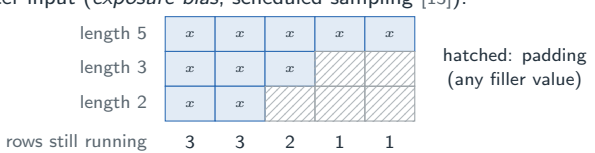
	largest $\|g\|$	final loss	diverged
no clipping	1.3×10^7	9.49	3 of 3
clipped, $\tau = 0.5$	1.6	1.86	0 of 3

- Unclipped, one exploding gradient wrecks the weights: the loss ends *above* the 4.17 of a uniform guess. Clipped, the same η trains normally.

Training II: teacher forcing, unequal lengths, dropout, optimiser



Teacher forcing: in training the input at step t is the true token y_{t-1}, whatever the model predicted. When generating, the model feeds on its own output: an early mistake (N for M) changes every later input (*exposure bias*; scheduled sampling [13]).



Unequal lengths: pad a batch to the longest sequence, then make sure the padding is never used: *pack* the batch (`pack_padded_sequence`: step t processes only the rows still running) or mask padded positions out of the loss.

Dropout in recurrent networks [14]

Apply dropout to the *vertical* connections (input → layer → layer → output), not to the recurrent connection $h_{t-1} \to h_t$: noise injected into the memory at every step accumulates and erases it. `nn.LSTM(dropout=p)` drops between stacked layers only; `SeqModel` adds dropout before the output layer.

Optimiser and learning rate

- Adam with η between 10^{-3} and 10^{-2} is a robust start for small recurrent models. Always clip (slide 12).
- `fit` restores the weights with the lowest validation loss.

Settings behind the results (Adam, clipping at 1)

	adding	sunspots	Shakespeare
hidden size n_h	32	32	256
sequence / chunk length	10–80	24	64
batch size	32	64	32 streams
learning rate	0.01	0.002	0.003
state between batches	reset	reset	carried

Outputs and losses: forecasting a number, predicting a token

Only the head on top of h_t and the loss depend on the task.

Regression head (forecasting)

$\hat{y} = w^\top h_T + b$: one linear output on the last hidden state. Loss: mean squared error $\frac{1}{N}\sum(\hat{y} - y)^2$ on standardised values. Report RMSE and MAE in original units ($\hat{y}\,\sigma + \mu$). For H steps ahead, train one model per horizon ("direct", used here) or feed predictions back in ("recursive"; errors accumulate).

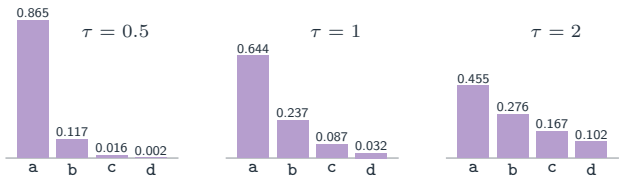
Softmax head (next-token prediction)

$z_t = W_y h_t + b_y \in \mathbb{R}^V$: the *logits*, one score per vocabulary entry, at every step.

$$p_t = \mathrm{softmax}(z_t), \qquad \mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \ln p_t[y_t] \quad \text{(cross-entropy)}$$

Perplexity $= e^{\mathcal{L}}$: the effective number of equally likely choices per token. Uniform over $V = 65$ characters: $\mathcal{L} = \ln 65 = 4.17$, perplexity 65.

By hand: true tokens got probabilities $(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{2})$.
$\mathcal{L} = \frac{1}{4}(0.693 + 1.386 + 2.079 + 0.693) = \mathbf{1.213}$, perplexity **3.36**.



$\mathrm{softmax}(z/\tau)$ for logits $z = (2, 1, 0, -1)$ over four tokens a–d.

Generation: sample, feed back, repeat

1. Run the prompt through the network; keep the state (h, c).
2. Sample the next token from $\mathrm{softmax}(z_t/\tau)$.
3. Feed that token in as the next input: one step, state updated. Go to 2.

- **Temperature** τ rescales the logits before the softmax. $\tau < 1$ sharpens the distribution (safer, more repetitive); $\tau > 1$ flattens it (more varied, more errors); $\tau \to 0$ is greedy $\arg\max$.
- If the true token above is `b`: loss $= -\ln 0.237 = \mathbf{1.44}$. Temperature is a sampling knob only; the loss is always computed at $\tau = 1$.

Code: the LSTM from scratch, forward and backward (NumPy)

`lstm_numpy.py`: one time step for a batch; `z` is stacked as [input, forget, candidate, output], the layout of `torch.nn.LSTM`

```
def lstm_step(x, h_prev, c_prev, Wx, Wh, b):
    n_h = h_prev.shape[1]
    z = x @ Wx.T + h_prev @ Wh.T + b                    # (B, 4 n_h)
    i = sigmoid(z[:, 0 * n_h:1 * n_h])                  # input gate
    f = sigmoid(z[:, 1 * n_h:2 * n_h])                  # forget gate
    g = np.tanh(z[:, 2 * n_h:3 * n_h])                  # candidate values
    o = sigmoid(z[:, 3 * n_h:4 * n_h])                  # output gate
    c = f * c_prev + i * g                              # keep some of the old, add some of the new
    tc = np.tanh(c)
    h = o * tc
    cache = (x, h_prev, c_prev, i, f, g, o, tc)
    return h, c, cache
```

Backpropagation through time, the loop of `lstm_backward`; agrees with finite differences and with PyTorch autograd to 10^{-10}

```
    for t in reversed(range(T)):
        x, h_prev, c_prev, i, f, g, o, tc = caches[t]
        dh = dH[:, t] + dh_next                         # from the output at t and from step t+1
        dc = dc_next + dh * o * (1 - tc ** 2)           # h = o * tanh(c)
        dz = np.concatenate([dc * g * i * (1 - i),      # through the input gate   (sigmoid' = s(1-s))
                             dc * c_prev * f * (1 - f), # through the forget gate
                             dc * i * (1 - g ** 2),     # through the candidate    (tanh' = 1 - tanh^2)
                             dh * tc * o * (1 - o)],    # through the output gate
                            axis=1)                     # (B, 4 n_h)
        dWx += dz.T @ x                                 # shared weights: gradients add over time
        dWh += dz.T @ h_prev
        db += dz.sum(axis=0)
        dX[:, t] = dz @ Wx
        dh_next = dz @ Wh                               # to h_{t-1}: through a weight matrix
        dc_next = dc * f                                # to c_{t-1}: only an elementwise product
```

Code: the training loop and the sampler (PyTorch)

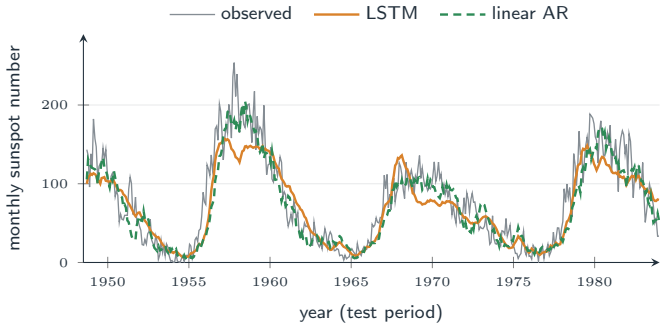
`train.py`, inside `fit()`: one loop for all three tasks (lines verbatim; logging and early stopping omitted)

```
    for step in range(1, steps + 1):
        x, y, first = next(batches)
        state = detach(state) if stateful and not first else None   # keep the values, cut the graph
        model.train()
        out, state = model(x, state)                          # forward through all T steps
        loss = seq_loss(out, y)
        opt.zero_grad()
        loss.backward()                                       # backpropagation through time
        norm = torch.nn.utils.clip_grad_norm_(model.parameters(), clip if clip > 0 else float("inf"))
        opt.step()
        losses.append(loss.item())
        norms.append(norm.item())                             # gradient norm BEFORE clipping
```

`train.py`, `generate()`: sampling with temperature; only the state carries the past

```
@torch.no_grad()
def generate(model: SeqModel, cd: CharData, prompt: str, n: int, temperature: float = 1.0,
             seed: int = 0) -> str:
    """Feed the prompt, then repeatedly sample one character and feed it back in."""
    model.eval()
    gen = torch.Generator().manual_seed(seed)
    out, state = model(cd.encode(prompt)[None])               # logits (1, len(prompt), V) and the state
    ids = []
    for _ in range(n):
        p = torch.softmax(out[0, -1] / temperature, dim=-1)   # temperature < 1 sharpens, > 1 flattens
        nxt = torch.multinomial(p, 1, generator=gen)          # sample one id
        ids.append(nxt.item())
        out, state = model(nxt[None], state)                  # one step: only the state carries the past
    return prompt + cd.decode(ids)
```

Results I: forecasting sunspots, with baselines



6-month-ahead forecasts on the 424 test months, each made from the 24 months before it. LSTM: seed 0. The test period holds the largest cycle on record (peak 254; training maximum 239), so every model must extrapolate.

Setup (`python train.py --task sunspots [--horizon 6]`): window of 24 months → LSTM, $n_h = 32$, 4,513 parameters → linear head on h_{24}. Chronological 70/15/15 split, scaling from 1749–1913.

Baselines. **Persistence**: the last observed value. **Seasonal-naive**: the value 132 months earlier (the mean solar cycle is about 11 years). **Linear AR**: least-squares linear map from the same 24 values, 25 parameters.

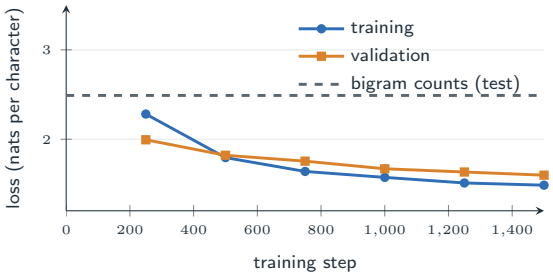
	1 month ahead		6 months ahead	
test set	RMSE	MAE	RMSE	MAE
persistence	19.7	14.6	30.7	23.9
seasonal-naive	46.9	33.8	46.9	33.8
linear AR(24)	**17.9**	13.2	**25.0**	18.9
simple RNN	$19.5_{\pm 0.3}$	14.0	$26.3_{\pm 1.2}$	19.9
GRU	$19.1_{\pm 0.2}$	14.0	$25.3_{\pm 0.4}$	19.2
LSTM	$19.3_{\pm 0.6}$	14.1	$26.6_{\pm 1.3}$	20.2

Sunspot-number units. Recurrent models: mean ± std over 3 seeds.

Honest reading

- Every learned model beats persistence and seasonal-naive.
- **The LSTM does not beat the 25-parameter linear model**: its RMSE is below linear AR in 0 of 3 seeds at 1 month and 0 of 3 at 6 months. Linear baselines are hard to beat on such series [19].
- RNN, GRU and LSTM are within noise of each other: a 24-step window holds no long-range dependency for gates to protect.

Results II: a character-level language model on Tiny Shakespeare



test set (55,769 chars)	loss	perplexity	bits/char
uniform over 65	4.17	65.00	–
unigram counts	3.36	28.85	–
bigram counts	2.49	12.06	–
LSTM	**1.70**	**5.49**	2.46

`python train.py --task shakespeare`: embedding 32 $\rightarrow$ LSTM, $n_h = 256$ $\rightarrow$ softmax over 65; 315,745 parameters. 1500 steps of truncated BPTT (32 streams $\times$ 64 characters, 3.1 passes over 1,003,854 training characters), 138 s on one CPU core. Split by position: 90% train, 5% validation, 5% test. One run. Each training point is the mean over the preceding 250 steps.

LSTM sample, temperature 0.5 (prompt `ROMEO:`)

```
ROMEO: why charge and your lord,
Tell thy banish strend hours do but were may
That in this soul of him stands of write to make thee
I will not my father well you for my life.

LEONTES:
I will we will have a man of thy day.
```

LSTM sample, temperature 1.0

```
ROMEO: Porforms kind! Therefore ineshand,
Now gelsent Merciues, his treetle hath good being some perices
And of yor welpermances in difler nenemity
Where penty one a man in his galk!

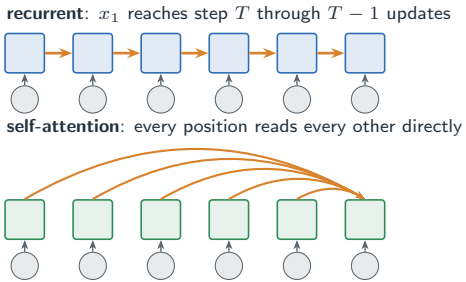
SICINIUS:
Why! But for the palief and love,--
```

Bigram baseline sample (one character of context; / marks a line break)

```
Ris remofed hexpselteredoca meaman youd INAlste grecofurey h / fotherere
hath thig berelisseanongilost. / tho y we w ke RYORD: / W
```

- The bigram table sees one character of context: perplexity 12.06. The LSTM carries context in its state and reaches 5.49: it has learned spelling, line lengths and the speaker-name layout.
- $\tau = 0.5$ gives mostly real words; $\tau = 1$ samples the model's own distribution and shows its errors.
- Validation loss is still falling: a two-minute CPU run, not a converged model.

Where LSTMs stand today, relative to transformers



per layer, length T, width d	recurrent	self-attention
computation	$O(T\,d^2)$	$O(T^2 d)$
sequential operations	$O(T)$	$O(1)$
path length between two positions	$O(T)$	$O(1)$
memory while generating	$O(d)$ state	$O(T\,d)$ cache

First three rows: Table 1 of [17].

Why transformers took over (2017 on): all positions of a training sequence are processed in parallel, and any two are one step apart: no long chain of Jacobians.

Rule of thumb: lots of data and offline training, use a transformer. Little data, a latency budget or a live stream: try a GRU or LSTM, and a linear baseline first.

Where recurrent models are still a sound choice

- **Streaming and low latency**: one input in, one output out, constant memory and cost per step, no context window to manage. On-device speech, sensor monitoring, control.
- **Small data**: a 5k-parameter LSTM trains in seconds on a CPU (slides 11, 17).
- **Time series**: attention shows no clear win; simple linear models have matched or beaten transformer forecasters [19], as the linear AR did here.

The ideas outlived the architecture

- The additive path of the cell state reappears as the residual connection in every transformer block.
- Recurrence is being revisited: xLSTM [18] changes the gating and memory of the LSTM and reports results competitive with transformers on language modelling. Independent large-scale confirmation is still limited.

References and how to run the code

[1] Elman. *Finding Structure in Time.* Cognitive Science 14(2), 1990.

[2] Werbos. *Backpropagation Through Time: What It Does and How to Do It.* Proceedings of the IEEE 78(10), 1990.

[3] Bengio, Simard, Frasconi. *Learning Long-Term Dependencies with Gradient Descent is Difficult.* IEEE Transactions on Neural Networks 5(2), 1994.

[4] Pascanu, Mikolov, Bengio. *On the Difficulty of Training Recurrent Neural Networks.* ICML 2013. arXiv:1211.5063

[5] Hochreiter, Schmidhuber. *Long Short-Term Memory.* Neural Computation 9(8), 1997.

[6] Gers, Schmidhuber, Cummins. *Learning to Forget: Continual Prediction with LSTM.* Neural Computation 12(10), 2000.

[7] Greff, Srivastava, Koutník, Steunebrink, Schmidhuber. *LSTM: A Search Space Odyssey.* IEEE TNNLS 28(10), 2017. arXiv:1503.04069

[8] Jozefowicz, Zaremba, Sutskever. *An Empirical Exploration of Recurrent Network Architectures.* ICML 2015.

[9] Cho, van Merriënboer, Gulcehre, Bahdanau, Bougares, Schwenk, Bengio. *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation.* EMNLP 2014. arXiv:1406.1078

[10] Chung, Gulcehre, Cho, Bengio. *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling.* 2014. arXiv:1412.3555

[11] Schuster, Paliwal. *Bidirectional Recurrent Neural Networks.* IEEE Transactions on Signal Processing 45(11), 1997.

[12] Sutskever, Vinyals, Le. *Sequence to Sequence Learning with Neural Networks.* NeurIPS 2014. arXiv:1409.3215

[13] Bengio, Vinyals, Jaitly, Shazeer. *Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks.* NeurIPS 2015. arXiv:1506.03099

[14] Zaremba, Sutskever, Vinyals. *Recurrent Neural Network Regularization.* 2014. arXiv:1409.2329

[15] Graves. *Generating Sequences With Recurrent Neural Networks.* 2013. arXiv:1308.0850

[16] Karpathy, Johnson, Fei-Fei. *Visualizing and Understanding Recurrent Networks.* 2015. arXiv:1506.02078

[17] Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin. *Attention Is All You Need.* NeurIPS 2017. arXiv:1706.03762

[18] Beck, Pöppel, Spanring, Auer, Prudnikova, Kopp, Klambauer, Brandstetter, Hochreiter. *xLSTM: Extended Long Short-Term Memory.* 2024. arXiv:2405.04517

[19] Zeng, Chen, Zhang, Xu. *Are Transformers Effective for Time Series Forecasting?* AAAI 2023. arXiv:2205.13504

Data. Monthly sunspot numbers 1749–1983 (`jbrownlee/Datasets` on GitHub). Tiny Shakespeare (`karpathy/char-rnn`; character models as in [15,16]).

Run it with `python <file>` (NumPy, PyTorch; CPU only)

`test_lstm.py`	eight checks, incl. BPTT vs. finite differences
`walkthrough.py`	hand calculations (slides 3–5, 7, 8, 10, 12–14)
`lstm_numpy.py`	NumPy LSTM vs. `torch.nn.LSTM`
`train.py --task sunspots`	or `adding`, `shakespeare`; see `--help`
`experiments.py`	figures for slides 3, 8–12, 17, 18

Exercises. Train with `--forget_bias 0`. Swap `--model gru` into the character model.